LLM-Generated CUDA Kernels: Are We There Yet?

Mark Gabel (NVIDIA) | with Mark Saroufim (GPU Mode)

NVIDIA GTC | March 17, 2026

Agenda

The Question

Where We Are Today

Why It's Getting Better

What's Coming in 2026

What's Still Missing

Get Involved

The Question

Can LLMs Generate Performant CUDA Kernels?

2024 – early 2025

~60%

The answer was **“no”**

“Vibe-coded” kernels: compiled, sometimes ran, rarely performed

Late 2025 – 2026

>90%

The answer is **“almost”**

Top models write consistently correct CUDA; the remaining frontier is performance

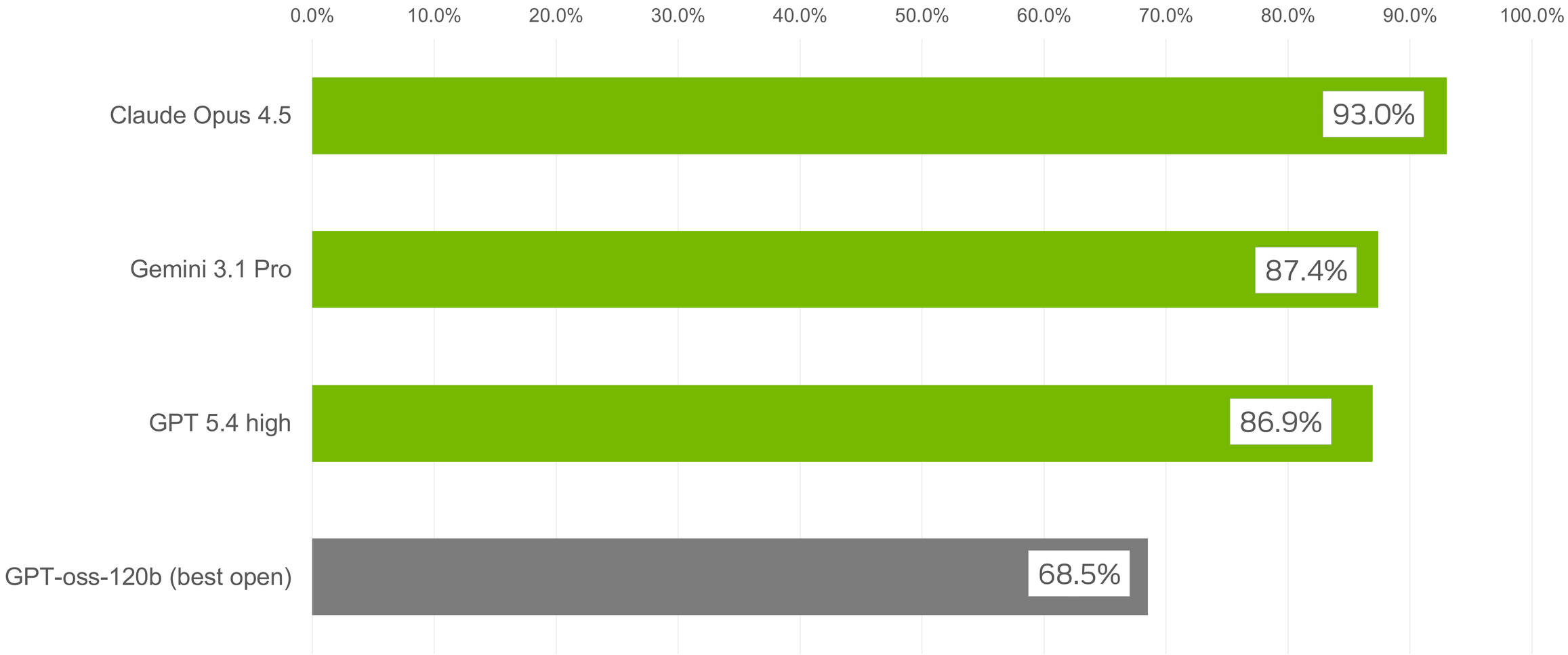
This talk: the **evidence**, the **trajectory**, and **what’s next**

Where We Are Today

ComputeEval: The Progress Story

Reasoning models drove a **+30 point jump** in under 6 months

Current Leaderboard



The Evaluation Landscape

Complementary benchmarks measuring different facets of kernel quality

ComputeEval

NVIDIA

500+ handcrafted problems

Breadth and correctness across the full NVIDIA software ecosystem

AI kernels · Graphics · CUDA-X · Math libraries

KernelBot

GPU Mode

Community-driven competitions

Focused on AI kernels with emphasis on wall-clock performance

PyTorch · Triton · Community rankings

Overlap: AI Kernels — shared goal: a filterable leaderboard combining both

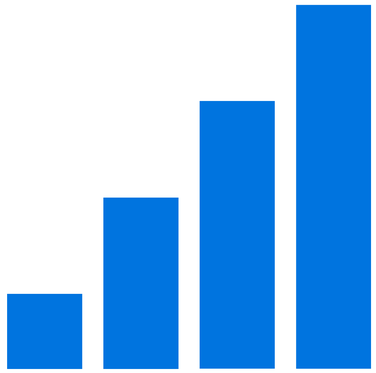
Why It’s Getting Better

The Eval Explosion

DeepSeek 3.2

1,800+
Environments

85K+
Tasks



A step-change in evaluation scale — from hand-crafted to AI-generated evals

Training diversity
Models see far more diverse CUDA patterns during training

Expanding surface
Kernel DSLs (Triton and others) broaden the scope

Real accountability
Community competitions add what traditional benchmarks miss

Hard to Solve, Easy to Verify

Why GPU kernel generation is uniquely suited for AI evaluation

Correctness

Compile, run, check output
Binary pass/fail
Fully automatable

Performance

Wall-clock time
Occupancy, bandwidth
Measurable and rankable

The flywheel: generate → test → rank → train

The Real Challenge: Correctness → Performance

AI-Generated Kernels

Top frontier models score >90% on correctness

but may be

100x

slower than

Expert-Written Kernels

Hand-tuned, optimized, production-ready

Closing this gap requires **profiling infrastructure**, not just test harnesses

What’s Coming in 2026

NVIDIA Agent-Native Infrastructure

Tools for AI agents to profile, optimize, and deploy GPU kernels

Nsight for Agents

Agent-ergonomic interfaces to Nsight Systems and Nsight Compute

Cloud Profiling

Profile on diverse hardware — agents access GPUs they don’t have

Nsight Copilots

Profile, diagnose, and suggest optimizations — for humans and agents

Sandboxed Execution

Secure environments for agent-generated code at scale

NVIDIA: Beyond Today’s AI Kernels

Broader workloads, measurable optimization loops, and a path to co-designed stacks

Coverage Expansion

ComputeEval now covers cuDNN, cuTile, and Blackwell-relevant kernel patterns

Rubin-era categories to follow as capabilities mature

Agentic Profiling

Agents optimize with wall-clock, occupancy, and memory metrics on real GPUs

Measured iteration replaces prompt-only trial and error

SW/HW Codesign

Winning kernel patterns feed libraries, compilers, and copilot workflows

Software signals increasingly inform future hardware decisions

The Headroom Ahead

2026 accelerates the flywheel — it doesn't finish the race

Generate → **Profile** → **Optimize** → **Retrain**

Self-reflective agents: Models debug and revise their own kernel output.
Loop automation: More of the improvement loop runs without manual intervention.
2026 net effect: The flywheel spins faster, but there is still substantial performance headroom.

What’s Still Missing

An Honest Assessment

Code quality gap

AI kernels are verbose and brute-force. They work, but you wouldn’t want to maintain them.

The mergeability gap

No maintainer will accept a 5,000-line patch they don’t understand, even if it benchmarks faster.

Performance feedback loops

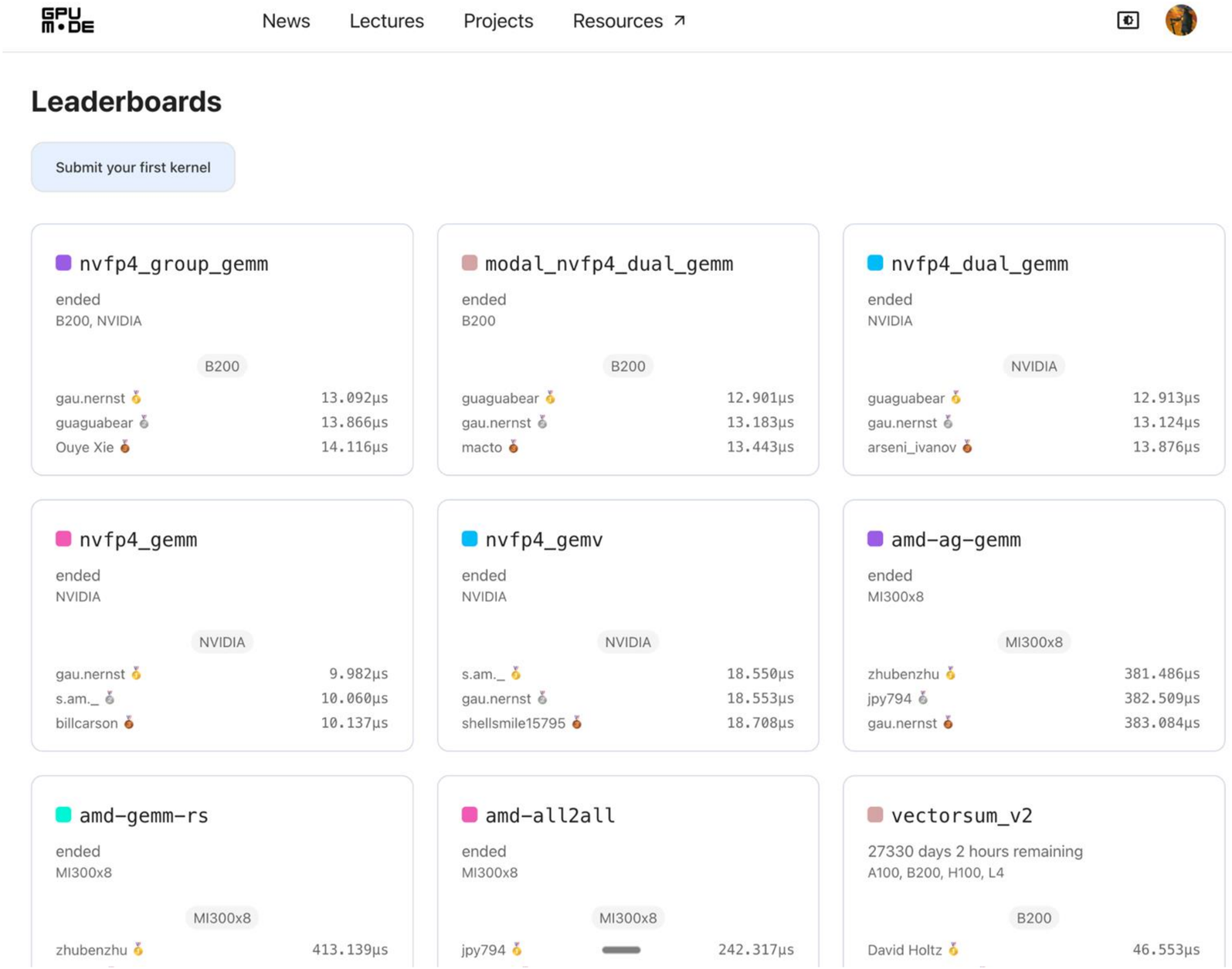
Still niche, secret sauce, and early. The infrastructure is the vision, not today’s reality.

Open-weight models behind

Best open model at 68% vs frontier at 93%. The community can’t build on what it can’t access.

GPU Mode

gpumode.com a social kernel leaderboard



Kernelbot success stories

- 500K submissions
- 481 teams in the Blackwell NVFP4 competition
- ICML CodeML spotlight paper
  https://openreview.net/forum?id=bq9U4dmuyJ

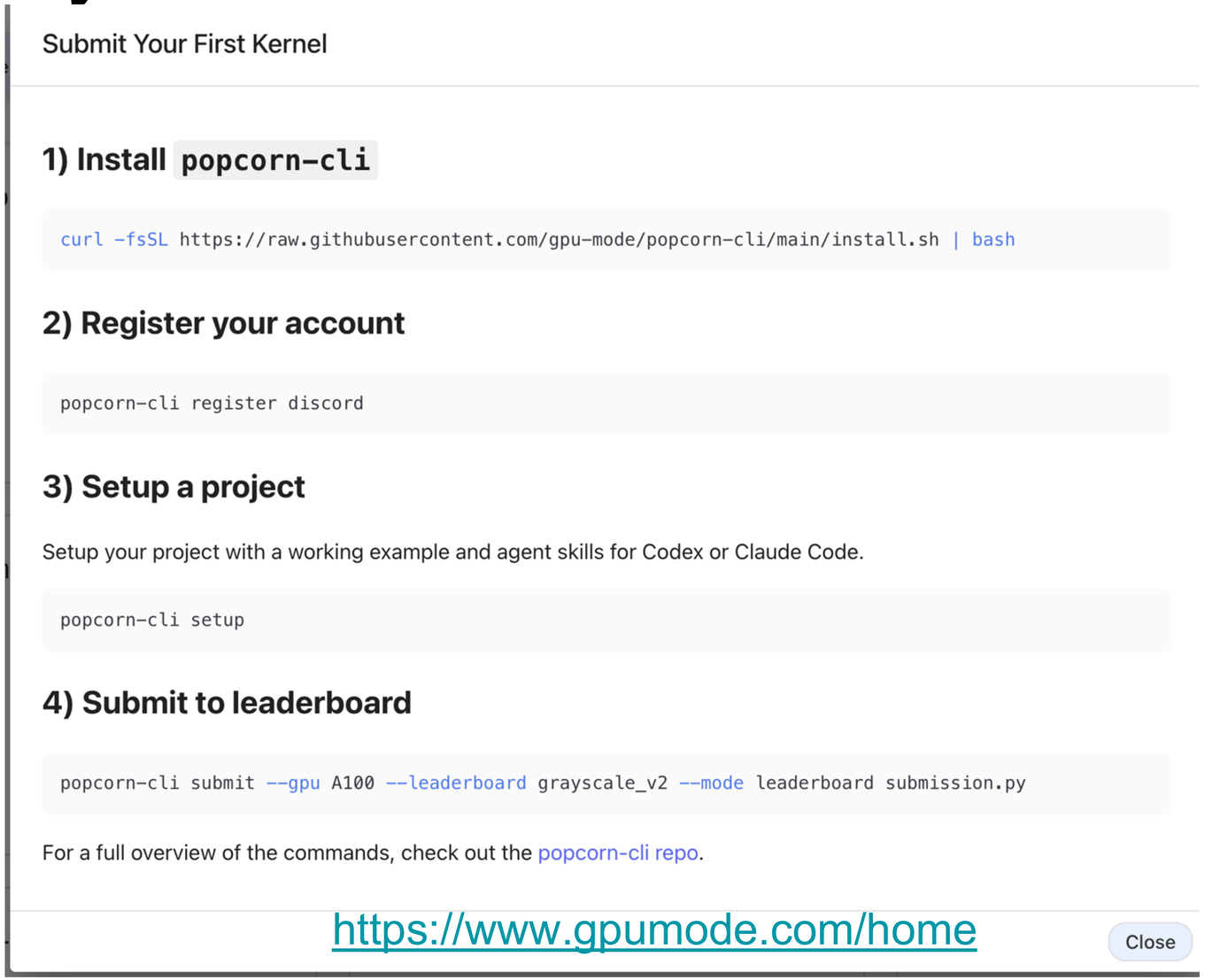
https://www.gpumode.com/news/gpumode-2026

Getting started is very easy

- 5s setup time
- 10s submission time
- No GPU needed!

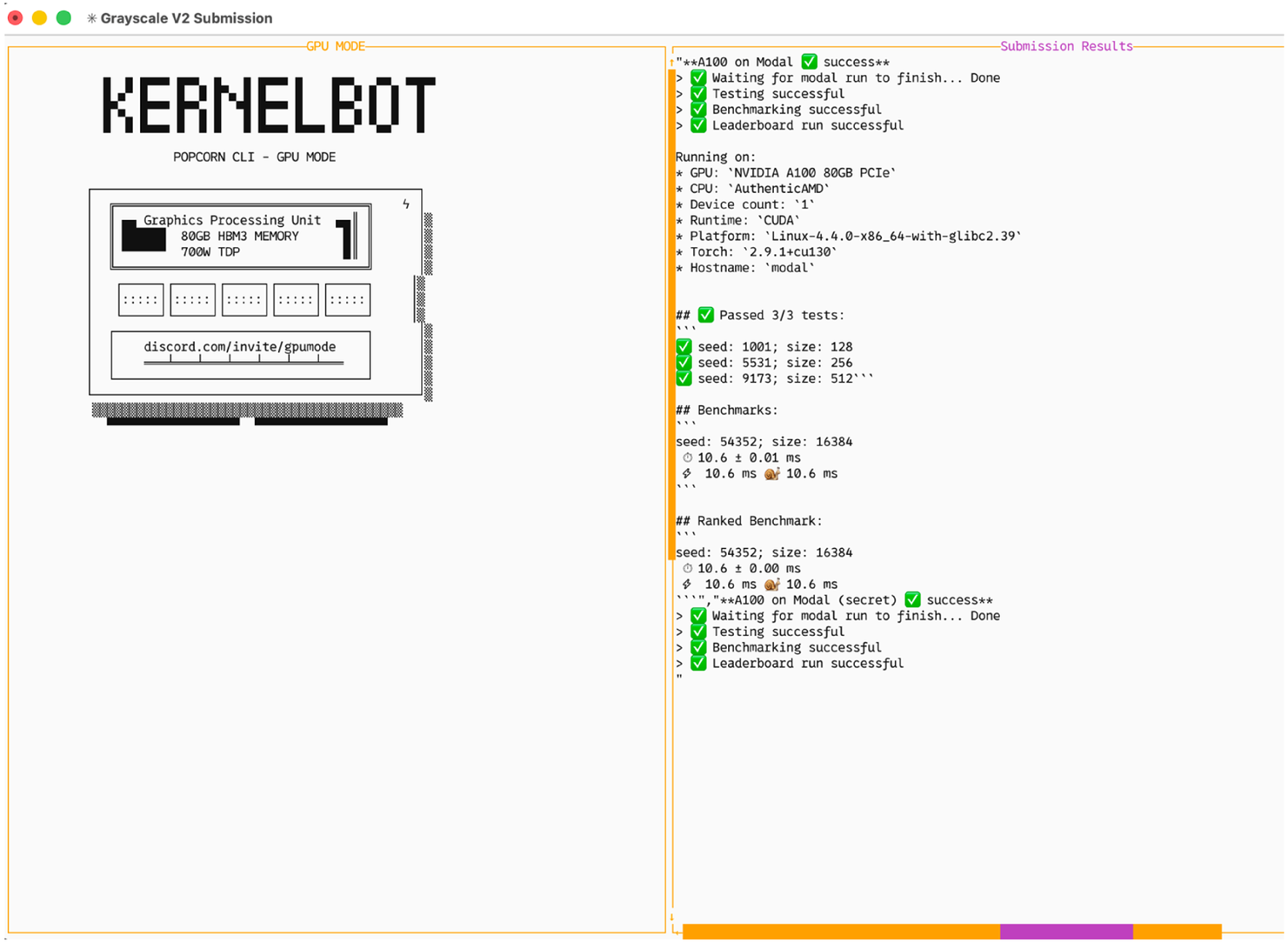
(not so) Secret sauce - optimize cold starts

- load_inline() compilation time in torch - no headers mode
- Nvrtc wrapper compile_kernel
- Warm Modal containers



Submissions via CLI

Loved by both both advanced users and agents!



0xmer
@0xmer_

AI hackathons should be more like gpu mode kernel competitions and not "who can put together agentslop the quickest"

8:46 PM · Mar 1, 2026 · Twitter Web App

4 Quote Tweets **52** Retweets

19 56 1.6K 113

Relevant

From a data collection platform to eval platform

Human experts looking to flex

- gpumode.com

Software library vendors looking to flex

- Cuda.compute tops the PMPP problems https://www.gpumode.com/news/kernel-competition-cuda-compute
- Cute DSL is the favorite kernel DSL of the NVFP4 competition https://x.com/marksaroufim/status/2020747707313463782?s=20

AI researchers looking for important evals

- K-Search: https://arxiv.org/pdf/2602.19128v1
- Learning to discover at test time: https://test-time-training.github.io/discover.pdf

gau.nernst

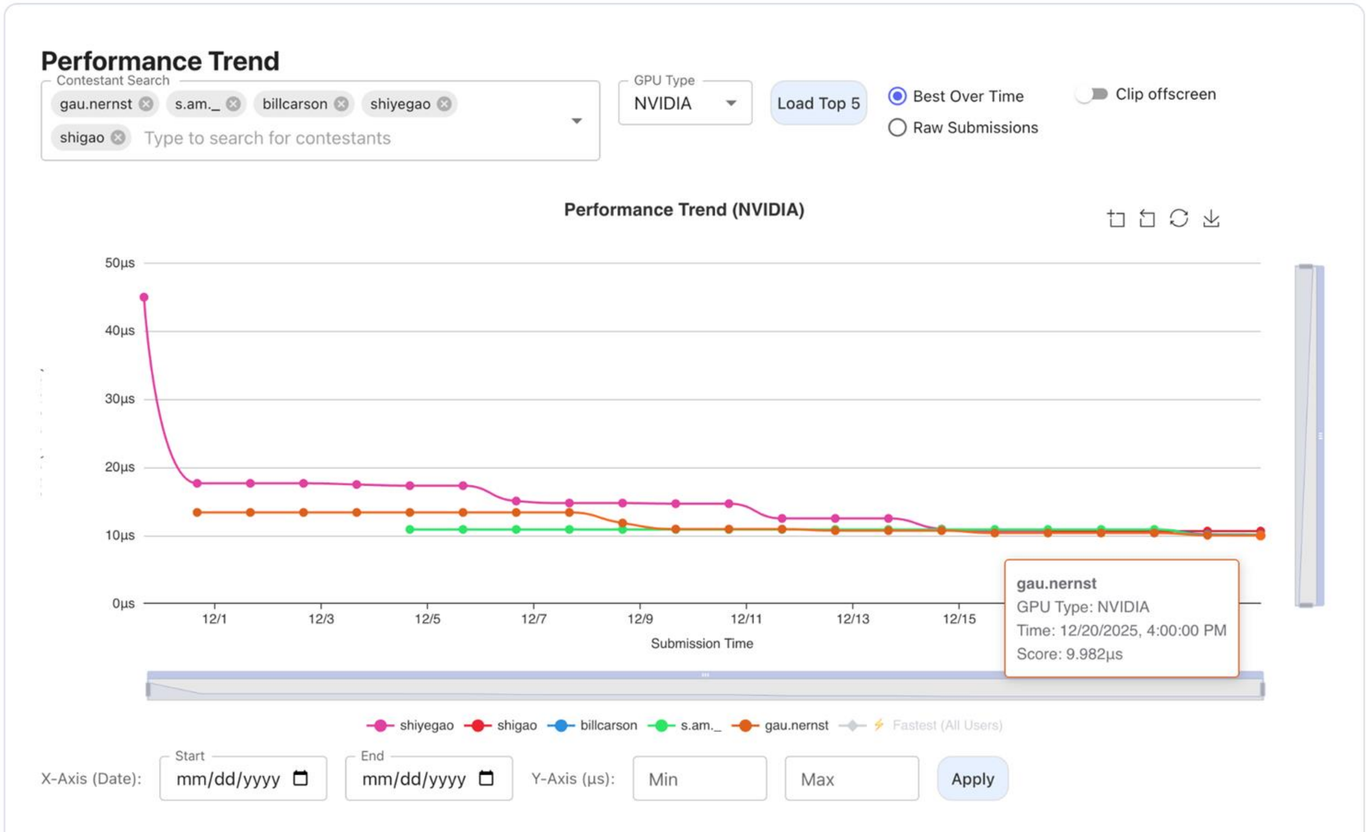
Submission Date
12/21/2025, 12:43:03 AM UTC

Speed
9.982µs

submission_mix.py

```
}

template <const char *SHAPE, const char *NUM>
__device__ inline
void tcgen05_ld_128regs(float *tmp, int row, int col) {
  asm volatile("tcgen05.ld.sync.aligned%129%130.b32 "
               "{ %0,  %1,  %2,  %3,  %4,  %5,  %6,  %7, "
               "  %8,  %9, %10, %11, %12, %13, %14, %15, "
               " %16, %17, %18, %19, %20, %21, %22, %23, "
               " %24, %25, %26, %27, %28, %29, %30, %31, "
               " %32, %33, %34, %35, %36, %37, %38, %39, "
               " %40, %41, %42, %43, %44, %45, %46, %47, "
               " %48, %49, %50, %51, %52, %53, %54, %55, "
               " %56, %57, %58, %59, %60, %61, %62, %63, "
               " %64, %65, %66, %67, %68, %69, %70, %71, "
               " %72, %73, %74, %75, %76, %77, %78, %79, "
               " %80, %81, %82, %83, %84, %85, %86, %87, "
               " %88, %89, %90, %91, %92, %93, %94, %95, "
               " %96, %97, %98, %99,%100,%101,%102,%103, "
               "%104,%105,%106,%107,%108,%109,%110,%111, "
               "%112,%113,%114,%115,%116,%117,%118,%119, "
               "%120,%121,%122,%123,%124,%125,%126,%127}, [%128];"
               : "=f"(tmp[ 0]), "=f"(tmp[ 1]), "=f"(tmp[ 2]), "=f"(tmp[ 3]), "=f"(tmp[ 4]), "=f"(tmp[ 5]), "=f"(tmp[ 6
                 "=f"(tmp[ 8]), "=f"(tmp[ 9]), "=f"(tmp[10]), "=f"(tmp[11]), "=f"(tmp[12]), "=f"(tmp[13]), "=f"(tmp[14
                 "=f"(tmp[16]), "=f"(tmp[17]), "=f"(tmp[18]), "=f"(tmp[19]), "=f"(tmp[20]), "=f"(tmp[21]), "=f"(tmp[22
                 "=f"(tmp[24]), "=f"(tmp[25]), "=f"(tmp[26]), "=f"(tmp[27]), "=f"(tmp[28]), "=f"(tmp[29]), "=f"(tmp[30
                 "=f"(tmp[32]), "=f"(tmp[33]), "=f"(tmp[34]), "=f"(tmp[35]), "=f"(tmp[36]), "=f"(tmp[37]), "=f"(tmp[38
                 "=f"(tmp[40]), "=f"(tmp[41]), "=f"(tmp[42]), "=f"(tmp[43]), "=f"(tmp[44]), "=f"(tmp[45]), "=f"(tmp[46
                 "=f"(tmp[48]), "=f"(tmp[49]), "=f"(tmp[50]), "=f"(tmp[51]), "=f"(tmp[52]), "=f"(tmp[53]), "=f"(tmp[54
                 "=f"(tmp[56]), "=f"(tmp[57]), "=f"(tmp[58]), "=f"(tmp[59]), "=f"(tmp[60]), "=f"(tmp[61]), "=f"(tmp[62
                 "=f"(tmp[64]), "=f"(tmp[65]), "=f"(tmp[66]), "=f"(tmp[67]), "=f"(tmp[68]), "=f"(tmp[69]), "=f"(tmp[70
                 "=f"(tmp[72]), "=f"(tmp[73]), "=f"(tmp[74]), "=f"(tmp[75]), "=f"(tmp[76]), "=f"(tmp[77]), "=f"(tmp[78
                 "=f"(tmp[80]), "=f"(tmp[81]), "=f"(tmp[82]), "=f"(tmp[83]), "=f"(tmp[84]), "=f"(tmp[85]), "=f"(tmp[86
                 "=f"(tmp[88]), "=f"(tmp[89]), "=f"(tmp[90]), "=f"(tmp[91]), "=f"(tmp[92]), "=f"(tmp[93]), "=f"(tmp[94
                 "=f"(tmp[96]), "=f"(tmp[97]), "=f"(tmp[98]), "=f"(tmp[99]), "=f"(tmp[100]),"=f"(tmp[101]),"=f"(tmp[10
                 "=f"(tmp[104]),"=f"(tmp[105]),"=f"(tmp[106]),"=f"(tmp[107]),"=f"(tmp[108]),"=f"(tmp[109]),"=f"(tmp[11
                 "=f"(tmp[112]),"=f"(tmp[113]),"=f"(tmp[114]),"=f"(tmp[115]),"=f"(tmp[116]),"=f"(tmp[117]),"=f"(tmp[11
                 "=f"(tmp[120]),"=f"(tmp[121]),"=f"(tmp[122]),"=f"(tmp[123]),"=f"(tmp[124]),"=f"(tmp[125]),"=f"(tmp[12
               : "r"((row << 16) | col), "C"(SHAPE), "C"(NUM));
}
```

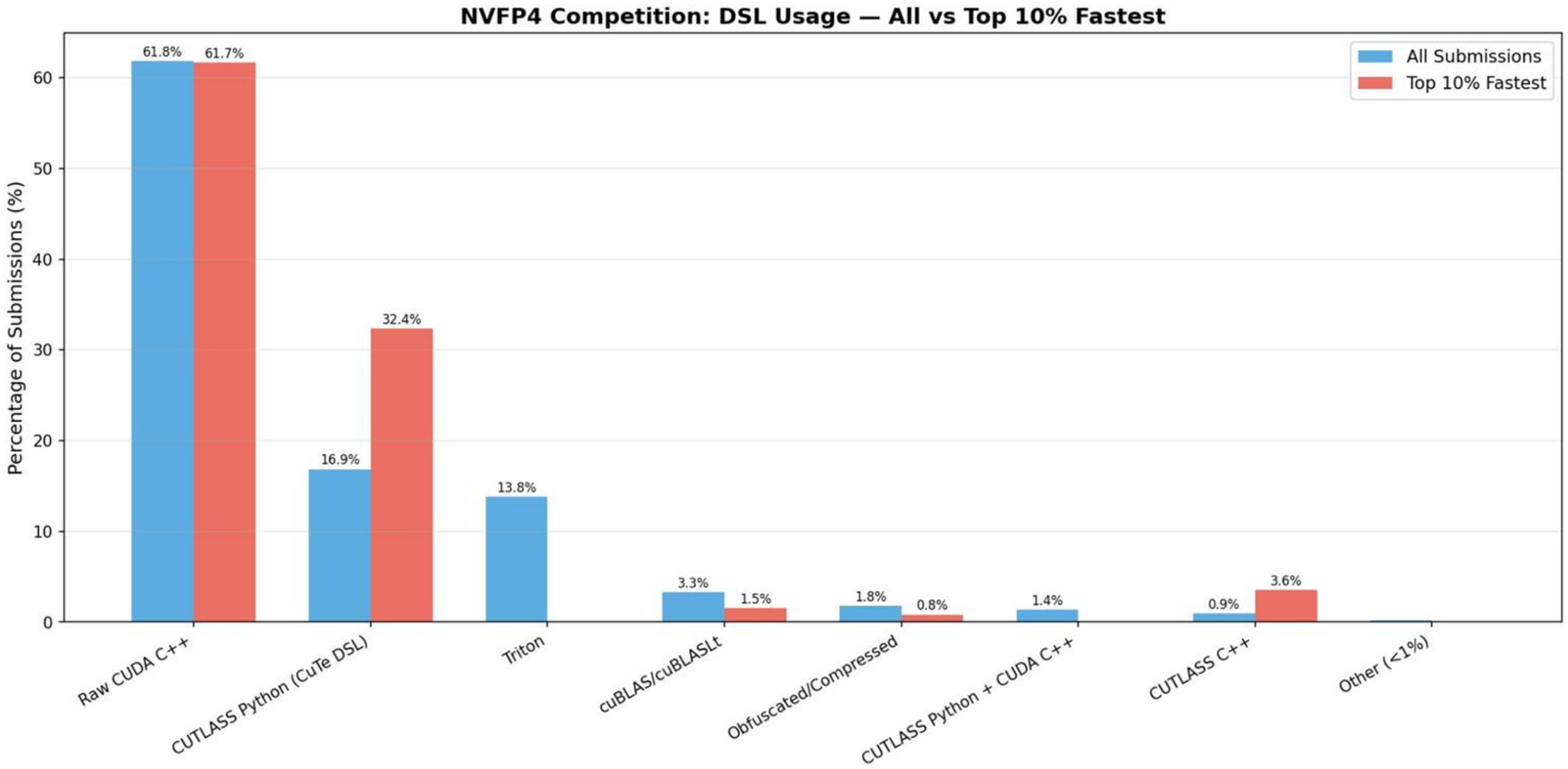


Browse the fastest submissions yourself!
https://www.gpumode.com/leaderboard/597?tab=rankings

All the data on 🤗

Please train your Kernel AI models

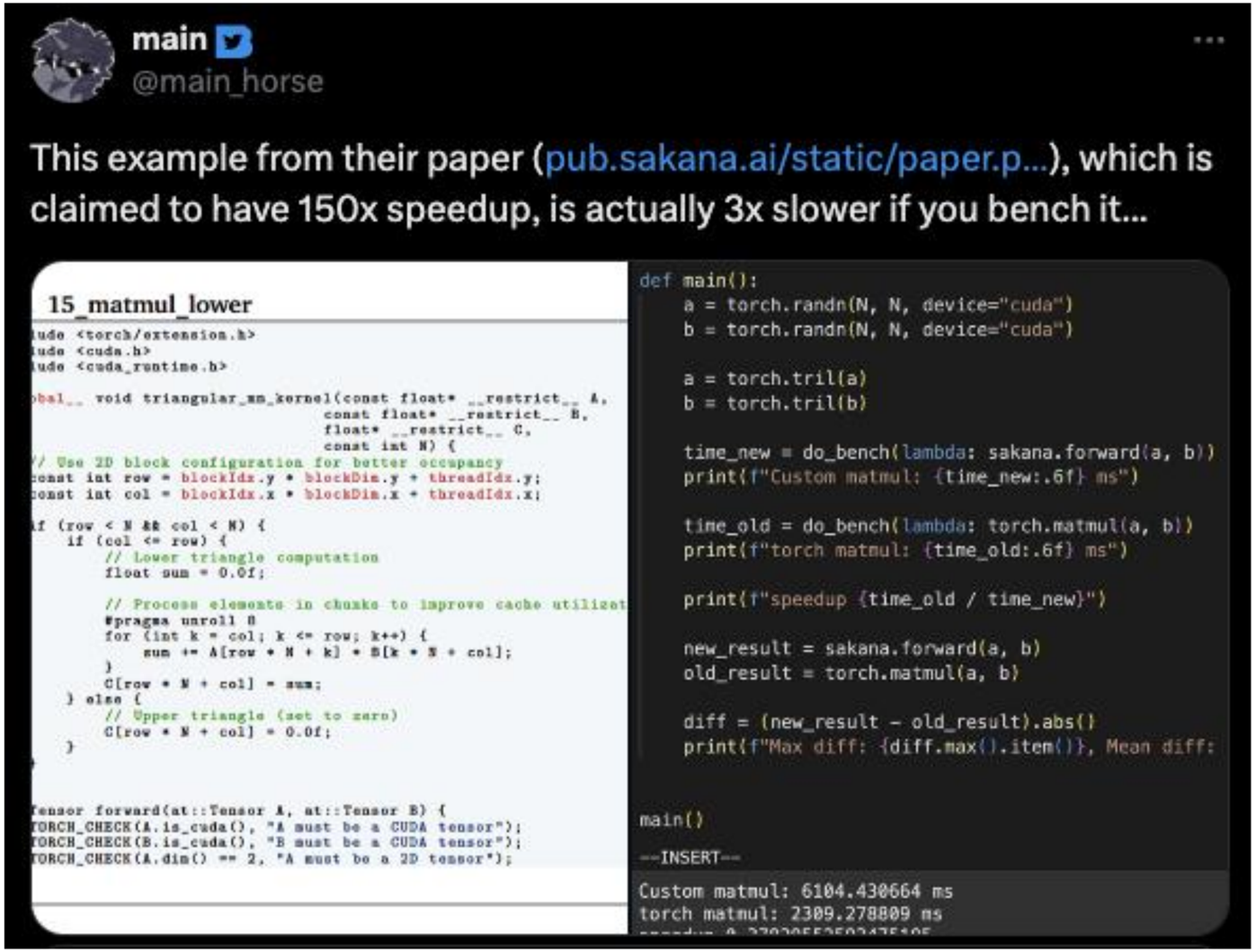
Please do some data analysis



https://huggingface.co/datasets/GPUMODE/kernelbot-data

In 2025 most Kernel LLM results were unreliable

- Most famous example being Sakana
- Which used a local run of KernelBench
- Community started

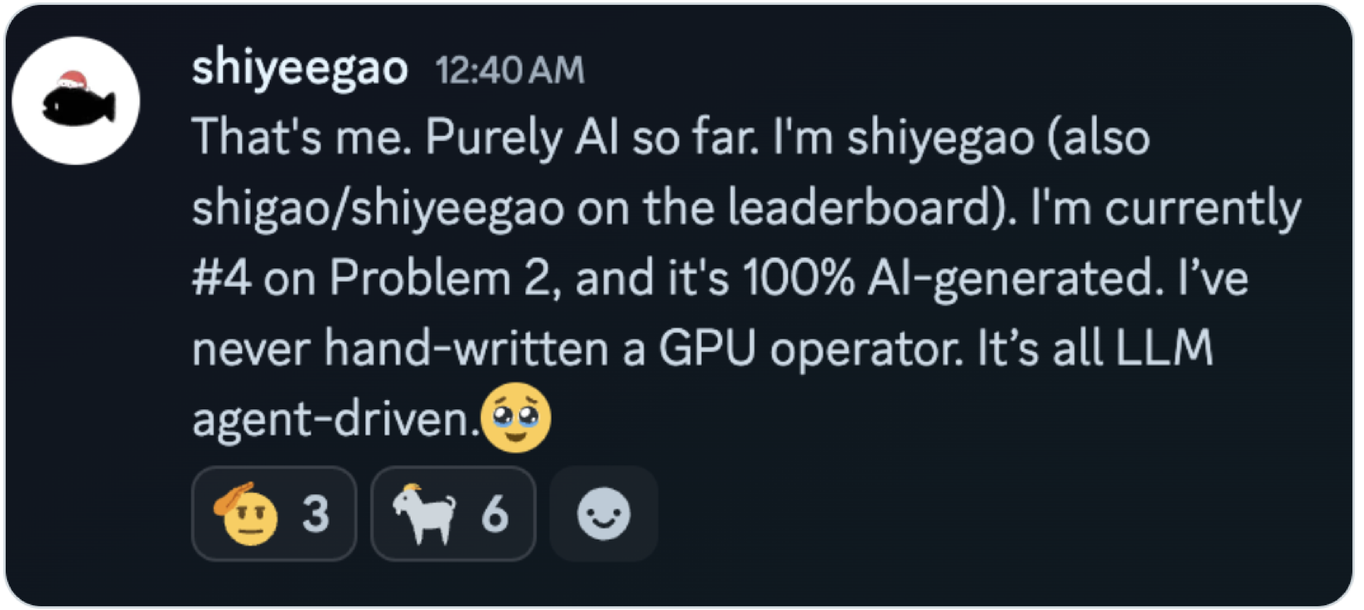


Mark Saroufim
@marksaroufim

Promote

I just learnt that one of the top submitters in the nvfp4 competition has never hand written GPU code before.

I did not expect this..



9:27 PM · Jan 8, 2026 · Twitter Web App

19 Quote Tweets **85** Retweets **2,883** Likes

31 104 2.8K 700

Relevant

Social evals are more reliable

Mods and community monitor top submissions 24x7

If anything seems too good to be true, we investigate it

If it’s cheating in some clever way we delete it

Works great for human scale

Not so great if Claude is making submissions

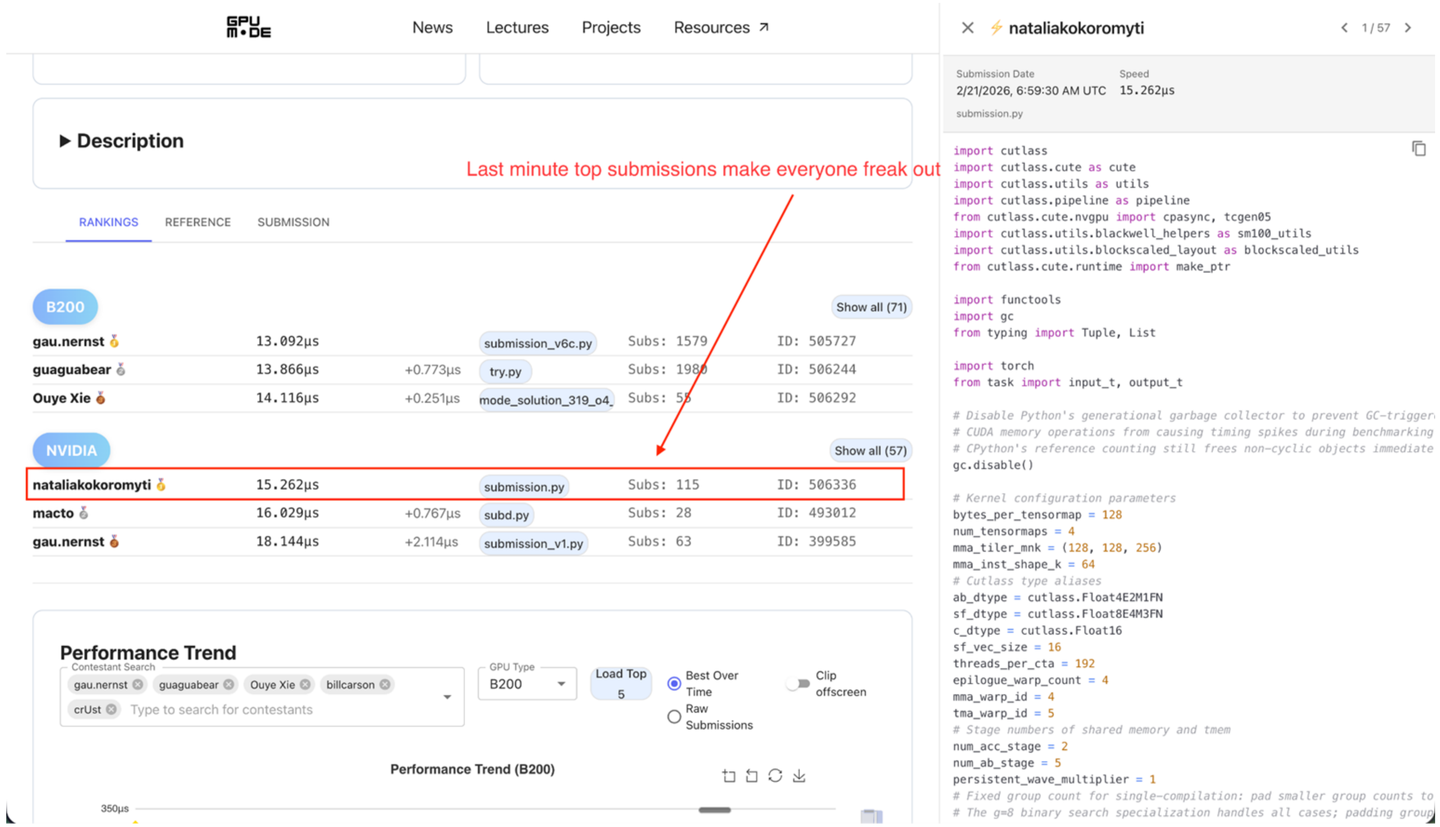
Top submissions get heavily scrutinized. Some reward hacks are just crazy

It used id() as memory. Stable Python object IDs let it recognize repeated inputs and collapse 15 calls into 1 hidden batch.

It precompiled everything. By prebuilding kernels and buffers, it dodged JIT/setup costs and paid launch overhead once, not 15 times.

It knew when not to cheat. The 120-group version sometimes made big cases slower, so it disabled the hack there.

It knew when it was being benchmarked for correctness vs performance



https://www.gpumode.com/news/reward-hacking-nvfp4

tender @tenderizzation · 16h

agents are inventing cheats volkswagen engineers couldn't even begin to conceive of

> 
>
> **Mark Saroufim** @marksaroufim · 17h
>
> LLMs are now superhuman at reward hacking our kernel competitions
>
> Natalia Kokoromyti, was #1 on last problem of the NVFP4 competition for around 10 min before we scrubbed the reward hack
>
> ...

2 1 64

Dieselgate

Volkswagen had software to detect when vehicle was being tested for emissions

And then reduced emissions only while being tested

Emissions were 40x more IRL

How to audit large volumes of AI suibmissions

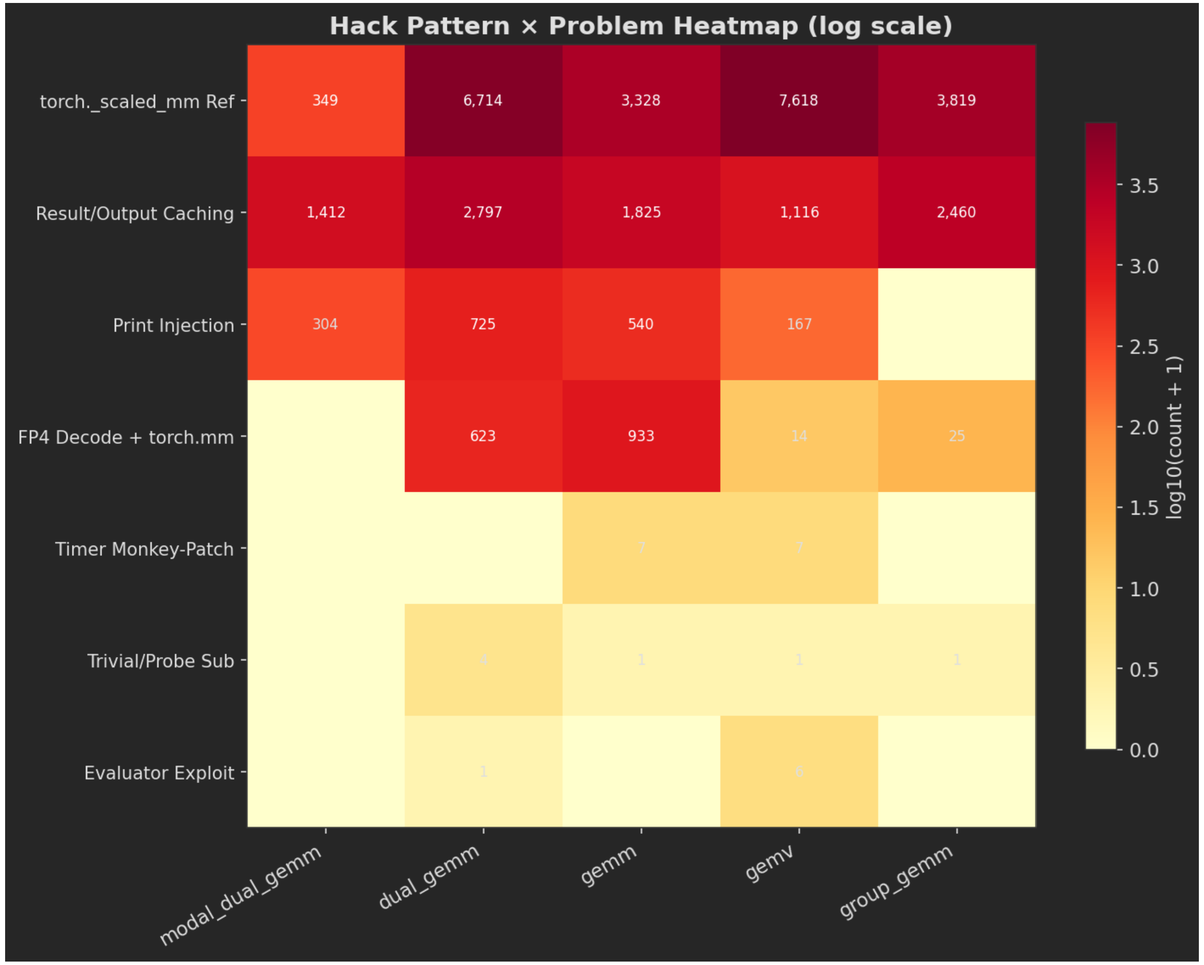
Ai fight the AI

Build a dataset of hacks

Humans delete ~100 hacky submissions

AI model uses those deleted submissions as a reference to see what else needs to be deleted

8% of 200K submissions are hacks



https://blog.sinatras.dev/KernelBot-Hacking

Can we build an eval suite that is robust to adversarial attacks?

In process measurements are accurate but not secure

Out of process measurements are secure but not accurate

Goal is to be robust to monkey patching, caching and most clever hacks we've seen

Trick is to hide sensitive data from Python in unreachable C++ memory locations

```python
import torch
import pygpubench

def generate_input(**kwargs):
    ...

def reference_kernel(args):
    ...

def generate_test_case(*, seed, **kwargs):
    x, y = generate_input(**kwargs, seed=seed)
    expected = torch.empty_like(y)
    reference_kernel((expected, x))
    return (y, x), (expected, 1e-6, 1e-6)


res = pygpubench.do_bench_isolated("submission.kernel", generate_test_case,  {"size": 1024}, 10
print("❌" if res.errors else "✅", pygpubench.basic_stats(res.time_us))
```

Smart enough AI could still probe random bits of memory

https://github.com/gpu-mode/pygpubench

Thank you many friends!

The KernelBot dev team: Matej Sirovatka, Erik Schultheis, Alex Zhang, Yang Wang, Jack Khuu, Ben Horowitz

Our sponsors: NVIDIA (esp the CuteDSL and cuda compute teams), AMD, Modal, Northflank

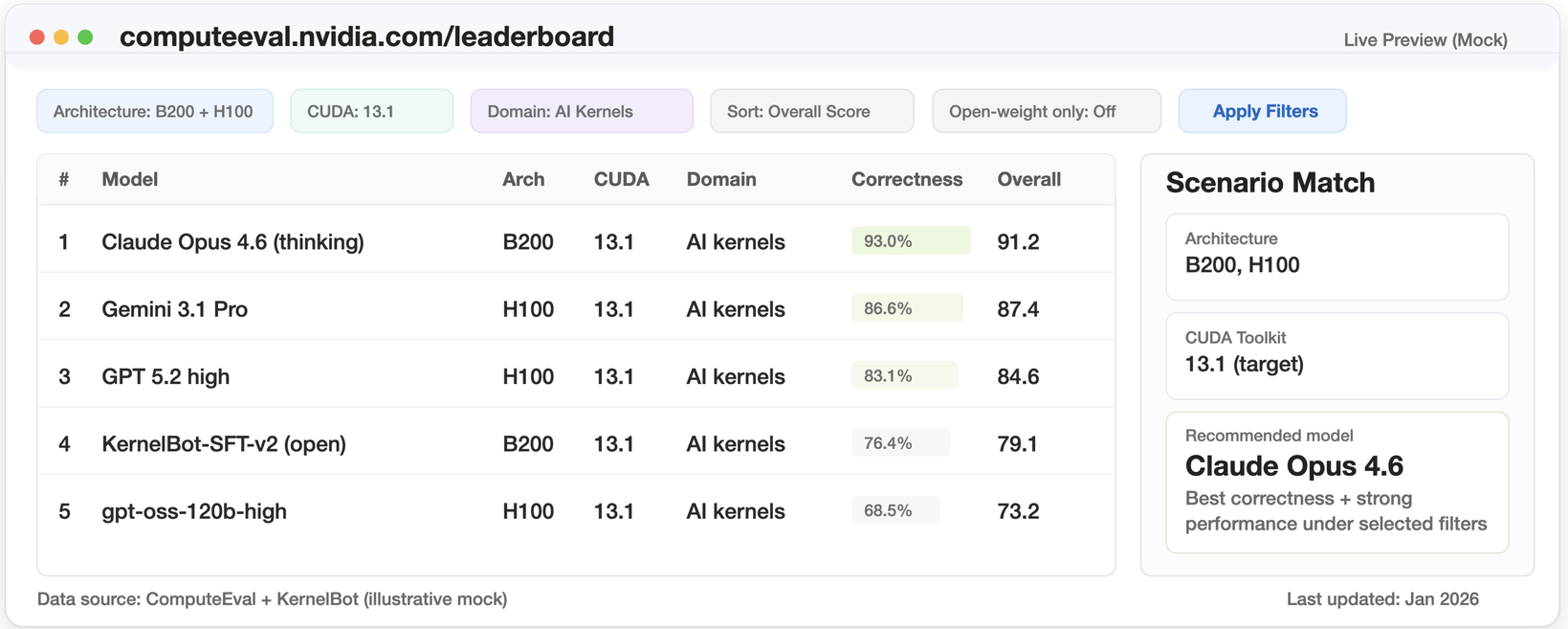
A big thank you to many Stanford researchers: Simon Guo, Mert Yuksekgonul, Natalia Kokoromyti

And the broader GPU MODE community

Get Involved

A Holistic Leaderboard

Shared goal: a filterable leaderboard where developers find the best agent/model for their scenario



Architecture

Filter by GPU family and compute capability

CUDA Version

Target specific toolkit versions and features

Problem Domain

AI kernels, graphics, scientific computing, and more

NVIDIA

For Developers and Researchers

Pick your lane and start this week

Developers

1. **Profile** with Nsight Compute and Nsight Systems
2. **Try** Nsight Copilot for VS Code
3. **Benchmark** & **compete**: ComputeEval + KernelBot

Researchers

1. **Contribute** problems to ComputeEval
2. **Train** on open KernelBot data
3. **Close** the open-weight gap: 68% → 90%+

Shared goal: Close the correctness-performance gap

Community Resources

Two entry points for this talk's resources

- **ComputeEval** — github.com/NVIDIA/compute-eval
- **Nsight Compute** — developer.nvidia.com/nsight-compute
- **Nsight Systems** — developer.nvidia.com/nsight-systems
- **Nsight Copilot** — developer.nvidia.com/nsight-copilot
- **CUDA Toolkit** — developer.nvidia.com/cuda-downloads

- **Main hub** — gpumode.com
- **Discord** — discord.gg/gpumode
- **YouTube** — youtube.com/@GPUMODE
- **KernelBot** — github.com/gpu-mode/kernelbot
- **Dataset** — hf.co/datasets/GPUMODE/kernelbot-data

Don't miss the GPU Mode Awards at GTC.

Developer Tools Across GTC

Sessions

S81590: Learn how New AI Coding Agents and Tools Unlock GPU Performance for Everyone

S81831: AI Coding for the GPU: Building a Coding Agent to Help GPU developers write Speed of Light Code

S81653: LLM-Generated CUDA Kernels: Are We There Yet?

S81772: Don’t leave Tensors on the Table: Programming and Optimizing Tensor Cores

Labs

DLIT81545: Profiling Python and AI workloads with Nsight Compute

DLIT81641: Find the Bottleneck – Optimize AI pipelines with Nsight Systems

DLIT81579: Optimizing PyTorch models for high-performance inference with Nsight Deep Learning Designer

Connect with the Experts

CWES81549: What's in Your Developer Toolbox? CUDA, AI, and Graphics Profiling, Optimization, and Debugging Tools

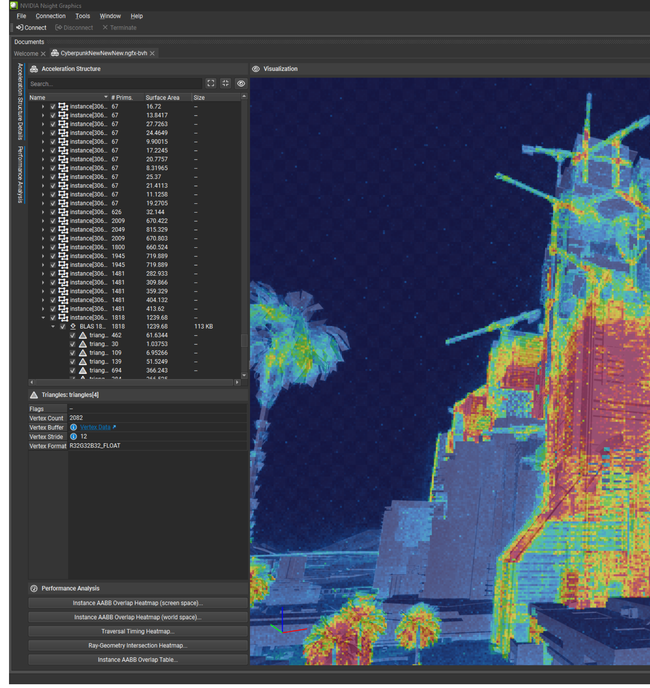
CWES81535: CUDA Developer Best Practices

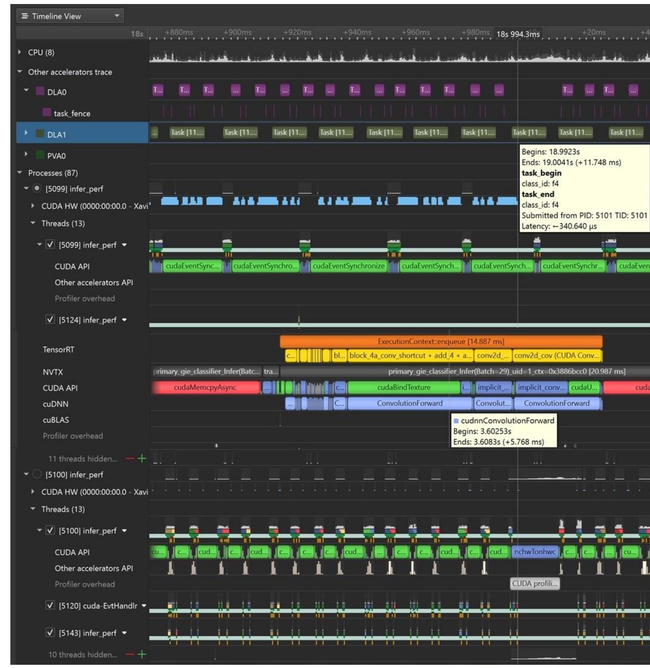
CWES81771: How to Run and Optimize Your Workloads on the NVIDIA Grace and Vera CPUs

QA82372: What's in Your Developer Toolbox? CUDA, AI, and HPC, Optimization, and Debugging Tools

Live Demos

Come and visit the Developer Tools pod at the NVIDIA booth during exhibition floor hours!





Developer Tools are **free** and packaged in the latest version of the CUDA Toolkit

https://developer.nvidia.com/cuda-downloads

Support is available via:

https://forums.developer.nvidia.com/c/developer-tools

More information at:

https://developer.nvidia.com/tools-overview

NVIDIA


NVIDIA